

```

1  #%%
2  import numpy as np
3  import pandas as pd
4  import matplotlib.pyplot as plt
5  import tensorflow as tf
6  from tensorflow import keras
7  from tensorflow.keras import layers, regularizers
8
9  print("TensorFlow version:", tf.__version__)


---


10 #%%
11 (train_data, train_labels), (test_data, test_labels
    ) = keras.datasets.imdb.load_data(
12     num_words=10000
13 )
14
15 def vectorize_sequences(sequences, dimension=10000):
16     results = np.zeros((len(sequences), dimension))
17     for i, sequence in enumerate(sequences):
18         results[i, sequence] = 1.0
19     return results
20
21 x_train = vectorize_sequences(train_data)
22 x_test = vectorize_sequences(test_data)
23 y_train = np.asarray(train_labels).astype("float32")
24 y_test = np.asarray(test_labels).astype("float32")
25
26 x_val = x_train[:10000]
27 partial_x_train = x_train[10000:]
28 y_val = y_train[:10000]
29 partial_y_train = y_train[10000:]
30
31 print("Training samples:", len(partial_x_train))
32 print("Validation samples:", len(x_val))
33 print("Test samples:", len(x_test))
34 print("Training data shape:", x_train.shape)
35 print("Test data shape:", x_test.shape)


---


36 #%% md
37
38 The function below makes it easier to run the same
    experiment with different numbers of hidden layers,
    hidden units, activation functions, loss functions,

```

```

38 dropout, and L2 regularization.
39 #%%
40 def run_model(
41     name,
42     hidden_units=(16, 16),
43     activation="relu",
44     loss="binary_crossentropy",
45     dropout_rate=None,
46     l2_rate=None,
47     epochs=10,
48     verbose=1
49 ):
50     model = keras.Sequential()
51     model.add(layers.Input(shape=(10000,)))
52
53     for units in hidden_units:
54         kwargs = {"activation": activation}
55         if l2_rate is not None:
56             kwargs["kernel_regularizer"] =
regularizers.l2(l2_rate)
57         model.add(layers.Dense(units, **kwargs))
58         if dropout_rate is not None:
59             model.add(layers.Dropout(dropout_rate))
60
61     model.add(layers.Dense(1, activation="sigmoid"))
62     model.compile(optimizer="rmsprop", loss=loss,
metrics=["accuracy"])
63
64     history = model.fit(
65         partial_x_train,
66         partial_y_train,
67         epochs=epochs,
68         batch_size=512,
69         validation_data=(x_val, y_val),
70         verbose=verbose
71     )
72
73     best_epoch = int(np.argmax(history.history["
val_accuracy"])) + 1
74     best_val = float(max(history.history["
val_accuracy"]))

```

```

75     test_loss, test_accuracy = model.evaluate(x_test
, y_test, verbose=0)
76
77     return {
78         "Model": name,
79         "Hidden Layers": len(hidden_units),
80         "Hidden Units": ", ".join(map(str,
hidden_units)),
81         "Change": name,
82         "Best Validation Accuracy": best_val,
83         "Best Epoch": best_epoch,
84         "Test Accuracy": float(test_accuracy),
85         "History": history.history
86     }


---


87 #%% md
88
89
90 The baseline uses two hidden layers with 16 units in
    each layer. The assignment asks for comparisons
    using one and three hidden layers and different
    numbers of hidden units.


---


91 #%%
92 results = []
93
94 experiments = [
95     ("One Hidden Layer", (16,), "relu", "
binary_crossentropy", None, None),
96     ("Baseline - 2 Layers", (16, 16), "relu", "
binary_crossentropy", None, None),
97     ("Three Hidden Layers", (16, 16, 16), "relu", "
binary_crossentropy", None, None),
98     ("32 Units", (32, 32), "relu", "
binary_crossentropy", None, None),
99     ("64 Units", (64, 64), "relu", "
binary_crossentropy", None, None),
100 ]
101
102 for exp in experiments:
103     result = run_model(*exp, verbose=1)
104     results.append(result)
105     print(f"{result['Model']}: validation={result['

```

```

105 Best Validation Accuracy']:.4f}, test={result['Test
Accuracy']:.4f}"))
106 ### md
107
108 MSE loss, tanh activation, dropout, and
    regularization.
109 ###
110 more_experiments = [
111     ("MSE Loss", (16, 16), "relu", "mse", None, None
112     ),
113     ("Tanh Activation", (16, 16), "tanh", "
    binary_crossentropy", None, None),
114     ("Dropout 0.2", (16, 16), "relu", "
    binary_crossentropy", 0.2, None),
115     ("Dropout 0.5", (16, 16), "relu", "
    binary_crossentropy", 0.5, None),
116     ("L2 Regularization", (16, 16), "relu", "
    binary_crossentropy", None, 0.001),
117 ]
118 for exp in more_experiments:
119     result = run_model(*exp, verbose=1)
120     results.append(result)
121     print(f"{result['Model']}: validation={result['
    Best Validation Accuracy']:.4f}, test={result['Test
    Accuracy']:.4f}"))
122 ### md
123
124 Validation accuracy is used to compare how the
    configurations performed during model development,
    while test accuracy provides the final performance
    on the held-out test set.
125 ###
126 results_df = pd.DataFrame(results)
127 display_cols = [
128     "Model", "Hidden Layers", "Hidden Units", "
    Change",
129     "Best Validation Accuracy", "Best Epoch", "Test
    Accuracy"
130 ]
131

```

```
132 results_display = results_df[display_cols].copy()
133 results_display["Best Validation Accuracy"] *= 100
134 results_display["Test Accuracy"] *= 100
135
136 results_display.columns = [
137     "Model", "Hidden Layers", "Hidden Units", "
    Change",
138     "Best Validation Accuracy (%)", "Best Epoch", "
    Test Accuracy (%)"
139 ]
140
141 display(results_display.round(2))
142 results_display.to_csv("IMDB_Neural_Network_Results.
    csv", index=False)
```

```
143 #%% md
144 Validation accuracy comparison
```

```
145 #%%
146 plt.figure(figsize=(12, 6))
147 plt.bar(results_display["Model"], results_display["
    Best Validation Accuracy (%)"])
148 plt.ylabel("Best Validation Accuracy (%)")
149 plt.xlabel("Model")
150 plt.title("IMDB Neural Network Validation Accuracy")
151 plt.xticks(rotation=45, ha="right")
152 plt.ylim(80, 92)
153 plt.tight_layout()
154 plt.show()
```

```
155 #%% md
156 Test accuracy Comparison
```

```
157 #%%
158 plt.figure(figsize=(12, 6))
159 plt.bar(results_display["Model"], results_display["
    Test Accuracy (%)"])
160 plt.ylabel("Test Accuracy (%)")
161 plt.xlabel("Model")
162 plt.title("IMDB Neural Network Test Accuracy")
163 plt.xticks(rotation=45, ha="right")
164 plt.ylim(80, 92)
165 plt.tight_layout()
166 plt.show()
```